Buffer World

A 3D Spatial Tutorial for Real Vim Motions

Concept and Technical Specification

Flyxion

Independent Researcher

July 2026

Contents

I Concept and Design **4**

1 Elevator Pitch **4**

2 Design Pillars **4**

3 The Core Translation: What Is 3D Space *Of*, Here? **4**

4 Mode System as World State **5**

5 The Verb–Noun Grammar as the Core Combat/Puzzle Loop **5**

6 Advanced Systems **6**

7 Level & World Structure **6**

8 Scoring: A Par System **6**

9 Technical Approach **7**

10 Open Design Questions **7**

11 What This Adds Over the Original **7**

II Technical Specification **7**

-1.1 A 3D spatial tutorial for real Vim motions — v0.3, merged with the concept sketch into a single LaTeX document 8

0 Non-Negotiables **8**

1 Formal Command Grammar **8**

2 Mode State Machine **9**

3 Buffer Model **10**

3.1 Word Classification 11
3.2 Undo Tree 11
3.3 Command Objects and Replay Semantics 11
3.4 Cursor Invariants 12

4 Motion & Operator Semantics (the part that must be exact) **13**

5 Text Objects **14**

5.1 Text Object Resolution 15

6 Rendering Mapping (renderer consumes parser state; never the reverse) **16**

6.1 Fidelity Principle 16

7 Level Data Format **16**

7.1 Pedagogical Layer 17

8 Testing Plan **17**
8.1 Differential Testing Against Real Vim 18

9 Build Phases **19**
9.1 Searchworks 19

10 Future Vim Coverage (Explicitly Out of Scope for v1) **20**

11 Global Commands and Substitute **21**
11.1 The Global Command Family 21
11.2 Substitute 22

12 User-Generated and Editable Levels **23**
12.1 Level Import Model 23
12.2 Author-Plays-First Par Seeding 23
12.3 Crowd-Sourced Par and Verified Replays 23
12.4 Repetition Detection and Macro-Friendliness 25
12.5 Placement in the Build Plan 25

Part I

Concept and Design

1 Elevator Pitch

Vim Adventures (2011) teaches `hjkl` and friends by turning a 2D tile grid into an overworld: you move the way you'd move the cursor, and the game only lets you progress by learning the next motion. This project asks what changes if the buffer isn't a flat grid but a navigable 3D space — where "the text" is something you walk through, climb, and reshape, rather than something you look at from directly above.

The goal stays disciplined: every mechanic maps to a **real, unmodified Vim command**. Nothing is vim-*flavored*; it's vim, wearing a world.

2 Design Pillars

1. **No invented commands.** If it's not a real Vim motion, operator, or mode, it doesn't go in the game. The temptation to add a "jump boot" power-up that isn't tied to an actual keystroke is the thing to resist hardest.
2. **The 3D-ness has to earn its keep.** A 3D game that plays exactly like the 2D original with better lighting isn't worth building. Section 3 below is the actual pitch: which Vim concepts *only* make sense once you have a third axis.
3. **Mistakes should look wrong, not just fail silently.** In real Vim, a mistyped command either does something confusing or nothing at all — the worst possible teacher. The game's job is to make the *consequence* of a command visible immediately, in space, before the player has to reason about it abstractly.
4. **Keystrokes stay real keystrokes.** No on-screen button mashing. The player types `3dw` on a real keyboard and watches it happen. The game is a skin over an actual modal input system, not a simulation of one.

3 The Core Translation: What Is 3D Space *Of*, Here?

This is the one decision everything else depends on, so it's worth spelling out three candidate mappings before picking one.

Option A — Depth as buffer stack. X/Y is the familiar row/column plane (rendered as a walkable floor grid, one tile per character), and Z is *which buffer or split you're in*. Switching windows (`Ctrl-w` family) becomes physically walking between elevated platforms; `:vsplit` cracks the floor open into a new platform beside you.

Option B — Depth as document structure. X/Y is still row/column, but Z encodes nesting — indentation level, or brace/paren depth. Diving into a function body means descending a level. `%` (jump to matching bracket) becomes a vertical teleport between the top and bottom of a shaft.

Option C — Depth as time (undo tree). X/Y is the buffer as usual; Z is the undo history, rendered as a tower you can walk down (`u`) and back up (`Ctrl-r`), with branches for `g-`/`g+` if the undo tree forked.

Recommendation: build A first, and treat B and C as world-specific level gimmicks rather than the permanent camera model. A is the least conceptually demanding — most new Vim

users don't yet think about splits, so making splits *physical* rather than abstract is a genuine teaching win. B and C are excellent later-level material precisely because they're harder ideas (nesting, undo trees) that benefit from being made concrete, but building the whole game around them first would front-load the hardest concepts.

4 Mode System as World State

Vim's biggest early confusion for new users isn't any single motion, it's that **the same key does different things depending on mode.** This is the single best argument for going 3D at all: mode can be a *visible property of the world*, not a mental flag the player has to track.

- **Normal mode** — default lighting, the player-character is solid, movement keys move the character.
- **Insert mode** — the world desaturates or dims at the edges, the character's "hands" visibly extend into the floor/wall in front of them (they're now writing into the structure itself), and `hjkl` stop moving the camera and start doing nothing (exactly as in real Vim) — which the game should let happen and let feel *wrong*, rather than intercepting it.
- **Visual mode** (`v`, `V`, `Ctrl-v`) — a glowing selection volume appears and grows as the player moves, previewing exactly what an operator will act on before they commit. This is the mode 3D helps most: `Ctrl-v` (visual block mode) is notoriously hard to picture in 2D tutorials and is trivial to picture as a literal 3D selection box.
- **Command mode** (`:`) — the world freezes and a console overlay drops down, visually distinct from gameplay, reinforcing that `:` commands are a different register of action (save, quit, search-and-replace) from buffer editing.

5 The Verb–Noun Grammar as the Core Combat/Puzzle Loop

Vim's real genius is that operators (`d`, `y`, `c`, `>`, `=`) compose with motions (`w`, `$`, `}`, `f x`) and text objects (`iw`, `a(`, `it`) into a small grammar that generates a huge command space from a few primitives. This grammar *is* the game's mechanical depth, and should be treated as such rather than smoothed over:

- **Operators** are "verbs" the player selects or holds (functionally: the first keystroke of a pending command). Visually, this could be a glow or particle effect anchored to the player, signaling "an action is armed and waiting for a target."
- **Motions and text objects** are "nouns" — literally the things in the world the glowing verb can be pointed at. `diw` (delete inner word) becomes: arm "delete," point it at a word-object, watch the word-platform vanish and the gap close.
- **Counts** (`3dw`, `5j`) become visible charge-up: a numeral appears in the HUD as it's typed, and the resulting action visibly happens N times in quick succession, so the player *sees* why `3dw` differs from `d3w` differs from `3d2w` (they're all the same in real Vim — 30, and this equivalence is a great early puzzle/gotcha for the game to demonstrate rather than explain).

This mirrors the same "verb applied to an addressable target" grammar already used elsewhere in your own work — text objects are exactly a *reach* into a structured space, the same shape as targeting a coordinate for a rendering request, just here the space being addressed is a text buffer instead of a provenance graph.

6 Advanced Systems

- **Registers** (`"ayy`, `"ap`) as an inventory system: named slots the player fills by yanking, and can later "spend" by pasting from. A level built around juggling three registers to rearrange three objects is a natural boss puzzle.
- **Marks** (`ma`, `` `a ``) as placeable waypoint beacons the player drops and teleports back to — useful the moment levels get large enough that walking back is tedious, which is also exactly when real Vim users start wanting marks.
- **Macros** (`qa ... q`, `@a`, `10@a`) as a "record a spell, then recast it" mechanic — record a sequence of actions once, then unleash it N times on a row of identical obstacles. This is probably the single most satisfying mechanic to render in 3D: watching a recorded macro auto-replay across a field of targets is inherently more dramatic in 3D than in a flat grid.
- **Search** (`/pattern`, `n`, `N`) as a radar/highlighting pulse that lights up matching objects across the visible world, with `n/N` snapping the camera to the next/previous hit.
- **Undo/redo** (`u`, `Ctrl-r`) as a visible rewind/fast-forward of the last action, not just a state change — the player should *see* the platform they just deleted rebuild itself.

7 Level & World Structure

Following the original's proven shape (new mechanic gated behind a small, forced-practice arena before it's allowed to mix with prior mechanics), but organized as a small set of "documents" (worlds) rather than one continuous overworld:

1. **The Plaza** — `hjkl`, `0/$`, `gg/G`. Pure movement, no operators yet.
2. **The Word Foundry** — `w/b/e/ge`, introduces the word as the atomic object the rest of the grammar will operate on.
3. **The Editing Yards** — `i/a/o/O`, first exposure to mode-switching, deliberately includes a puzzle solvable only by *escaping* insert mode (teaching that `Esc` is not optional).
4. **The Grammar Works** — operators (`d/y/c`) combined with everything learned so far. This is where the verb–noun system in Section 5 becomes the primary loop.
5. **Searchworks** — `:s/:%s` and `:g/:v`, the selection-and-transformation family that sits alongside operators and text objects rather than above them, since it depends on neither registers, marks, macros, nor undo. Placed here rather than later specifically because it doesn't need anything the later worlds teach. Every match highlights as a population simultaneously, not one at a time, and a substitution or `:g`-driven action resolves that whole population toward its result in one coordinated motion — the world's visual realization of match-set → operation → result, letting a player see the population before committing to transform it, the same way Visual mode previews an operator's effect before it lands.
6. **The Archive Stacks** — registers, marks, and the buffer-stack Z-axis from Option A above; splits and windows become physically real.
7. **The Loop Chamber** — macros, counts, and repeat (`.`), building toward puzzles only efficiently solvable by recording and replaying a macro.
8. **The Undo Spire** — the undo-tree-as-tower idea from Option C, offered as an optional late-game area for players who want the harder conceptual payoff rather than a mandatory main-path level.

8 Scoring: A Par System

Vim Adventures already implicitly rewards fewer keystrokes; making this explicit as a **par score per puzzle** (fewest keystrokes to solve, shown alongside the player's actual count) gives skilled players a reason to replay early levels with newly learned tricks — solving Level 1 in

3 keystrokes once you know `.` and counts, instead of the 12 it took the first time through, is its own kind of satisfying. This is the same competitive-efficiency shape as "pipeline golf" — judging a transformation by how few steps it takes to faithfully reach a target — just applied to keystrokes instead of rendering pipelines.

9 Technical Approach

- **Engine:** Three.js is the pragmatic choice for a browser-playable prototype — no install friction, and the mechanics here (grid-aligned movement, simple selection volumes, particle effects for verbs/macros) don't need a heavier engine.
- **Input layer:** the actual hard part is not the 3D rendering, it's a correct modal-keystroke parser — normal/insert/visual/command mode dispatch, pending-operator state, count-prefix accumulation, and the motion/text-object grammar. This should be built and tested as a standalone, headless module *before* any 3D work starts, so the "is this real Vim behavior" question is answered independently of how it's rendered.
- **World representation:** a buffer is a 2D array of characters plus metadata (word boundaries, indentation, bracket matching) computed once per edit; the 3D scene is a pure function of that array, regenerated on change rather than hand-animated per keystroke. This keeps the renderer honest — if the buffer state doesn't change, nothing in the world should either.

10 Open Design Questions

- How much of real Vim's *modal escape-hatch chaos* (pressing `Esc` twice, accidentally triggering `ZZ`, etc.) should the game protect against versus let happen? Protecting too much undercuts the "this is real Vim" pillar; protecting too little risks frustrating brand-new players in a way the original 2D game's simpler visual grammar didn't.
- Should text objects beyond `iw`/`aw` (e.g. `it`/`at` for tags, `i"`/`a"` for quotes) require the world to render visible tag/quote structure, or would that only make sense in a "the buffer is actual code" late-game world rather than the more abstract early plaza?
- Multiplayer/co-op is tempting given the macro-replay and register-passing mechanics (one player records, another triggers) but adds real complexity and isn't needed for the core teaching goal — worth treating as a stretch feature, not a v1 pillar.

11 What This Adds Over the Original

Not "better graphics" — the specific things a third dimension makes possible that a top-down grid cannot: mode as visible world-state rather than a tracked mental flag, visual-block mode as a literal volume instead of a description, splits/windows as physically real places instead of a toggled UI panel, and macros as something dramatic to *watch* replay across a field of targets rather than a log of keystrokes scrolling past.

Part II

Technical Specification

-1.1 A 3D spatial tutorial for real Vim motions — v0.3, merged with the concept sketch into a single LaTeX document

This supersedes the earlier concept sketch's prose sections with precise specifications, and is merged with it as a single document; heading numbers below are illustrative in this markdown source and are superseded by automatic numbering in the merged document.

---

0 Non-Negotiables

These are the correctness bar, not aspirations. A build that violates any of these has failed the project's actual goal, regardless of how the 3D world looks.

1. Every keystroke sequence the player can type must produce **exactly** the buffer state real Vim would produce for the same sequence on the same input, including the specific off-by-one and edge-case behaviors in Section 4.
2. The input parser (Section 2–4) must be implemented and unit-tested as a **headless module with zero rendering dependencies**, before any Three.js code is written. The renderer consumes parser output; it never influences parser behavior.
3. No command, mode, or key that doesn't exist in real Vim is ever bound to anything. Cosmetic-only extensions (camera shortcuts, menu keys) must use keys Vim itself doesn't bind, and must be documented as out-of-grammar.

---

1 Formal Command Grammar

A Normal-mode command, in the fragment this project targets for v1, is generated by the following grammar. This intentionally mirrors the style of a BNF specification rather than prose, for the same reason a prose description of Vim's grammar tends to hide exactly the ambiguities that matter.

```
<command>       ::= <prefix>* <operator-cmd> | <prefix>* <motion-cmd> | <prefix>*
↪  <action>
<prefix>        ::= <count> | "\"" <register>
<operator-cmd>  ::= <operator> <count>? ( <motion> | <text-object> )
                  | <self-apply>                        ; see self-application list
                  ↪  below
<motion-cmd>    ::= <motion>
<count>         ::= [1-9][0-9]*
<operator>      ::= "d" | "y" | "c" | ">" | "<" | "=" | "g~" | "gu" | "gU" | "!"
<self-apply>    ::= "dd" | "yy" | "cc" | ">>" | "<<" | "=="
                  | "guu" | "gugu" | "gUU" | "gUgU" | "g~~" | "g~g~"
<motion>        ::= "h" | "l" | "j" | "k"
                  | "0" | "^" | "$" | "gg" | "G"
                  | "w" | "W" | "b" | "B" | "e" | "E" | "ge" | "gE"
                  | "f" <char> | "F" <char> | "t" <char> | "T" <char> | ";" | ","
                  | "{" | "}" | "(" | ")" | "%"
                  | "H" | "M" | "L"
```

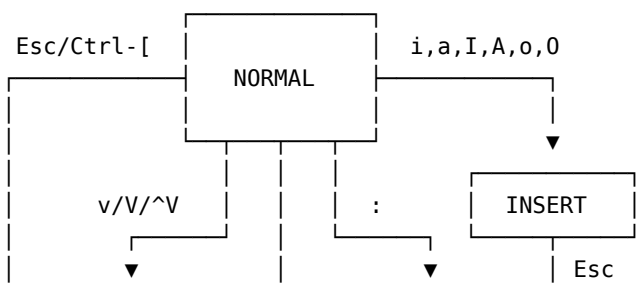
```
<text-object>   ::= ("i" | "a") <object-kind>
<object-kind>   ::= "w" | "W" | "(" | ")" | "b" | "{" | "}" | "B"
                  | "[" | "]" | "<" | ">" | "\"" | "'" | "t" | "p" | "s"
<action>        ::= "i" | "a" | "I" | "A" | "o" | "O"      ; enter Insert mode
                  | "v" | "V" | "\x16"                     ; enter Visual
                  ↪  (char/line/block)
                  | "x" | "X" | "~" | "r" <char>
                  | "u" | "\x12"                           ; undo / redo (Ctrl-r)
                  | "." | "p" | "P"
                  | "q" <register> ... "q"                 ; macro record
                  | "@" <register>
                  | "m" <mark> | "`" <mark> | "'" <mark>
                  | "/" <pattern> <CR> | "?" <pattern> <CR> | "n" | "N"
                  | ":" <ex-command> <CR>
```

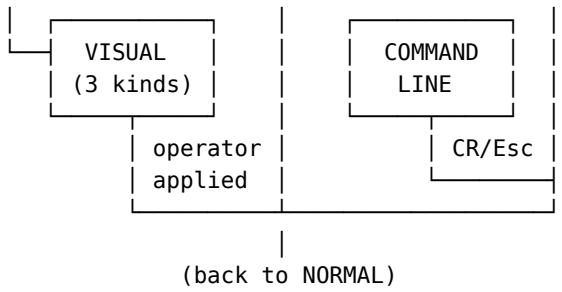
Three properties of this grammar drive most of the parser's complexity and are worth stating outside the grammar itself:

- **Register and count prefixes are order-independent.** The previous revision of this grammar attached the register prefix only to the outside of an already-complete command (`"a` followed by a whole `<operator-cmd>`), which cannot generate `"a3yy` and `3"ayy` as the same command — real Vim accepts both, with identical effect. The `<prefix>*` rule above fixes this: `<count>` and `" <register>` may appear in either order, zero or one of each, before the core command. If a count appears in a `<prefix>` and a second count appears later inside `<operator-cmd>` (e.g. `3"a2yy`), the two counts multiply per the existing count-multiplication rule below — `3"a2yy` yanks 6 lines into register `a`, the same as `"a6yy`. Resolution into a `ResolvedCommand` (Section 3.3) discards ordering entirely: the resolved `count` and `register` fields are the same regardless of which order they were typed in.
- **Count multiplication.** `<count>` may appear before the operator *and* before the motion in the same command, and the two multiply: `3d2w` deletes `3 × 2 = 6` words, identically to `d6w` or `6dw`. This is a real Vim behavior, not a simplification, and Section 5 lists it as a required test case precisely because it's the first thing an incorrect implementation gets wrong.
- **Operator self-application is linewise, and the v1 grammar fixes the full supported set explicitly** rather than leaving it as a "such as" list: `dd`, `yy`, `cc`, `>>`, `<<`, `==`, and the case-changing doubles `guu`/`gugu`, `gUU`/`gUgU`, `g~~`/`g~g~` (both the doubled-letter and doubled-operator spellings are accepted for the case operators, matching real Vim). Each is grammatically distinct from `<operator><motion>` and always means "apply linewise to `<count>` lines starting at the cursor line," not "apply the operator to itself as a motion." No other operator combination self-applies; `!!`, `d=`-style speculative doubles, and anything not in `<self-apply>` above are simply not valid commands.

---

2 Mode State Machine






```
  Additional transition, not drawn above to keep the box diagram legible:
  VISUAL (any kind) --Esc--> NORMAL, canceling the selection with no
  operator applied and no buffer change. This is a frequent, ordinary
  transition — not an edge case — and is listed here rather than boxed
  above only because a second exit arrow crowded exactly the corner of the
  diagram already carrying the "operator applied" path, making the
  diagram harder to read than it clarified.

  Sub-state of NORMAL, not a separate top-level mode:
  PENDING-OPERATOR — entered after an operator key with no motion yet
  (e.g. "d" alone). Accepts: count, motion, text-object, or the same
  operator again (linewise self-application). Times out to nothing on Esc.
```

State is represented as:

```
type Mode = "NORMAL" | "INSERT" | "VISUAL_CHAR" | "VISUAL_LINE"
          | "VISUAL_BLOCK" | "COMMAND_LINE";

interface ParserState {
  mode: Mode;
  pendingOperator: Operator | null;   // e.g. "d", null if none pending
  pendingCount: number | null;        // count typed before operator
  pendingRegister: string | null;     // "a in "ayy
  countBuffer: string;                // digits typed so far, this command
  visualAnchor: Position | null;      // fixed end of visual selection
  lastFindChar: { cmd: "f"|"F"|"t"|"T"; char: string } | null; // for ; and ,
  lastChange: RecordedCommand | null; // for "."
  recordingMacro: string | null;      // register name, or null
  macroRecording: KeyEvent[];
}
```

---

3 Buffer Model

```
interface Position { line: number; col: number; }  // 0-indexed internally

interface Buffer {
  lines: string[];                // one string per line, no trailing \n
  cursor: Position;
  marks: Map<string, Position>;   // 'a'-'z' local, "'" and "`" implicit marks
  undoTree: UndoNode;             // see Section 3.2
  currentUndoNode: UndoNode;
}

interface Register {
  text: string;
  kind: "charwise" | "linewise" | "blockwise";
```

```
}
// registers: Map<string, Register>, keys 'a'-'z' (append with 'A'-'Z'),
// '"' (unnamed, implicit target of any yank/delete without explicit register),
// '0' (last yank specifically), '1'-'9' (delete history ring)
```

3.1 Word Classification

Vim's `w/b/e` and `W/B/E` differ in exactly one respect: lowercase motions treat three character classes as boundaries (**word-chars** `[A-Za-z0-9_]`, **punctuation** `[^A-Za-z0-9_\s]`, and **whitespace**), while uppercase motions collapse word-chars and punctuation into one class (**WORD**, anything non-whitespace) and only whitespace is a boundary. This single classification function is shared by `w`, `b`, `e`, `ge`, and by the `iw/aw` text objects, so it should be implemented exactly once:

```
type CharClass = "word" | "punct" | "space";
function classify(ch: string): CharClass { /* per rule above */ }
```

3.2 Undo Tree

Vim's undo is a tree, not a stack — a new edit made after undoing does not delete the redo branch, it forks it, reachable via `g-/g+` (time-travel through *all* states in the order visited) or `u/Ctrl-r` (parent/last-child only). This is exactly the "World D — Undo Spire" content, and the data structure should be built as a tree from the start rather than retrofitted:

```
interface UndoNode {
  id: number;
  bufferSnapshot: string[];   // full snapshot; diffing is an optimization, not v1
  parent: UndoNode | null;
  children: UndoNode[];
  seq: number;                // global sequence number, for g-/g+ ordering
  timestamp: number;
}
```

3.3 Command Objects and Replay Semantics

Macros and `.` look similar from the outside — both "do something again" — but they replay at two different levels of representation, and conflating them is a reliable source of bugs.

`.` replays a resolved command, not keystrokes. The parser must retain a structured record of the last change, not the raw keys that produced it. The previous revision of this document gave `ResolvedCommand` a single shape built around `operator` and `target`, which has no room for count-prefixed insertions (`3ohello<Esc>`, `3ihello<Esc>`) or simple repeated actions (`3x`, `5~`) — neither has an operator or a motion/text-object target in the sense Section 1's grammar defines. The corrected shape is a tagged union:

```
type ResolvedCommand =
  | {
      kind: "operator-motion";
      operator: Operator;
      count: number;              // fully multiplied — 6, not "3" and "2" separately
      target: Motion | TextObject;
      insertedText?: string;      // present only for c-style changes
      register?: string;
    }
  | {
      kind: "insert-action";
      action: "i" | "a" | "I" | "A" | "o" | "O";
```

```
    count: number;
    insertedText: string;       // captured verbatim between entering Insert and Esc
  }
| {
    kind: "simple-action";
    action: "x" | "X" | "~" | "r" | "J" | "gJ";
    count: number;
    char?: string;               // present only for "r"
    register?: string;
  };
```

`3d2w` and `d6w` resolve to the identical `{kind: "operator-motion", operator: "d", count: 6, target: "w"}`, and it is this resolved form — not either surface spelling — that `.` replays, at whatever position the cursor currently occupies. This is also why `.` after a `c`-style change retypes `insertedText` verbatim rather than re-entering Insert mode for the player to retype.

Count semantics differ by insert-action, and this difference is not optional to get right. `i`, `a`, `I`, and `A` repeat the *inserted text in place*: `3ihello<Esc>` on an empty line produces a single line reading `hellohellohello` — the count multiplies how many times the same text is appended at the insertion point, not how many separate edits occur. `o` and `O` repeat the *whole open-line-and-insert operation*: `3ohello<Esc>` produces three new lines, each independently containing `hello`, because each repetition opens its own new line before inserting. A `ResolvedCommand` of `kind: "insert-action"` must therefore branch on `action` at replay time — the same `count` field means two structurally different replays depending on whether `action` is in `{i, a, I, A}` or `{o, O}` — rather than being interpreted uniformly the way `count` is for `operator-motion`.

`J`'s count means something different from every other simple action's count. For `x`, `X`, `~`, and `r`, `count` means "repeat this action N times" in the ordinary sense (`3x` deletes 3 characters). For `J`, `count` means "join this many lines total, counting the current line" — `3J` joins the current line with the next 2, not the current line with the next 3. This mirrors `G`'s count meaning "go to line N" rather than "repeat G N times" (Section 4), and should be implemented as the same kind of named special case, not derived from the general repeat-N-times rule.

A new count given at `.`-time replaces, not multiplies. If the player types a count immediately before `.` (e.g. `5.` following an earlier `3dw`), the stored count is *replaced* by 5 for this one replay — the result is `d5w`, not `d15w`, and the replacement does not persist: a bare `.` afterward reuses whichever count last stood, following the same replace-not-accumulate rule. This is a real, easily-missed Vim behavior and belongs in the Section 8 test suite as its own case.

Macros replay raw keystrokes, not resolved commands. A macro register (`qa ... q`) stores the literal `KeyEvent` sequence typed during recording, and `@a` re-feeds that sequence through the ordinary parser from scratch — it does not store or replay `ResolvedCommand` objects. This is why a macro built around `dw` deletes whatever word is actually under the cursor at each replay site, rather than a fixed span: the macro has no memory of what the first replay resolved to, only of which keys were pressed. `10@a` repeats the whole recorded sequence ten times, not any single command within it six or ten times.

3.4 Cursor Invariants

Most bugs in a from-scratch Vim clone are cursor bugs, not edit bugs — the edit itself lands correctly and the cursor ends up somewhere Vim would never put it. The invariants below should be enforced as a single clamping function called after every buffer mutation, not re-derived ad hoc per command.

1. `cursor.line` is always in `[0, lines.length - 1]`.

2. In `NORMAL`, `VISUAL_*`, and `COMMAND_LINE` modes, `cursor.col` is clamped to `[0, max(0, lineLength - 1)]` — the cursor may never rest one past the last character of a non-empty line. An empty line clamps to column 0.
3. In `INSERT` mode only, `cursor.col` may equal `lineLength` (one position past the last character), since that is where typed text would be appended. Leaving Insert mode re-applies rule 2 immediately, which is why the cursor visibly steps back one column on `Esc` at end-of-line — this is correct behavior to replicate, not a bug to fix.
4. After a linewise delete, the cursor is not preserved at its prior column; it moves to the first non-blank column of whichever line now occupies the cursor's old line index.
5. After a charwise delete that removes the character the cursor was on, the cursor clamps left under rule 2 if the deletion shortened the line below the old column.
6. `J` (join) leaves the cursor at the join point — the single space inserted between the two joined lines (or, for `gJ`, at the exact boundary, since `gJ` does not insert a space).
7. A mark referencing a line that has since been deleted is cleared, not left dangling. Jumping to a cleared mark (`` `a `` or `'a`) is a documented no-op, not an error condition the parser needs to surface.

---

4 Motion & Operator Semantics (the part that must be exact)

Every motion has a **type** that determines how an operator combined with it behaves. Getting this table wrong is the single most common way a from-scratch Vim clone feels almost-but-not-quite right.

```
type MotionKind = "exclusive" | "inclusive" | "linewise";
```

- **Exclusive** — the operator's range is `[start, destination)`: the character the motion lands the cursor on is not included.
- **Inclusive** — the range is `[start, destination]`: the destination character is included.
- **Linewise** — the range expands to whole lines regardless of the columns either endpoint's motion actually computed, from the first line touched through the last, newline characters included on both ends.

Motion	Type	Notes
`h`, `l`	exclusive	won't cross line boundaries
`j`, `k`	linewise	
`0`	exclusive	
`^`	exclusive	first non-blank
`$`	inclusive	**special-cased**: `d$` deletes to end of line *and* the type is inclusive, unlike most exclusive-by-default column motions
`w`	exclusive	**special-cased**: if `w` is the object of an operator and lands on the first non-blank of a new line, it's pulled back to end of previous line (`dw` at end of line does not delete the newline)
`W`	exclusive	same special case as `w`
`b`, `B`	exclusive	
`e`, `E`	inclusive	
`ge`, `gE`	inclusive	
`f{c}`, `t{c}`	inclusive	

Motion	Type	Notes
`F{c}`, `T{c}`	exclusive	
`gg`, `G`, `{count}G`	linewise	
`{`, `}`	exclusive	
`(`, `)`	exclusive	
`%`	inclusive	
`H`, `M`, `L`	linewise	

The `cw`/`ce` special case. By Vim's documented behavior, if the cursor is on a non-blank character, `cw` behaves like `ce` (changes to the end of the current word, not to the start of the next one) — the one deliberate exception carved out of the "operator + motion" composition rule, added specifically because `dw`'s "don't eat the trailing whitespace into the next word" behavior felt wrong for `c`. This must be implemented as an explicit special case, not derived from the general rule, because it isn't derivable from it.

Linewise operator range. When an operator combines with a linewise motion (`dj`, `d}` is *not* linewise — `}` is exclusive; `dG`, `dgg` *are*), the operator acts on whole lines, including the newline character, and the cursor lands on the first non-blank of the resulting line — not at the column it started on.

Required test cases (Section 8 formalizes this as an actual test suite; this is the minimum set that must pass before any 3D work begins):

```
buffer: "The quick brown fox"     cursor col 0
  dw   → "quick brown fox"
  de   → "The quick brown fox" minus "The" but with trailing space (differs from dw!)
  cw   → behaves as ce: enters insert with " quick brown fox" and cursor after "The"

buffer: "foo"                      cursor col 0  (word at end of buffer, no trailing
↪  word)
  dw   → "" (does not error, does not hang)

buffer: "foo\nbar"                 cursor at end of "foo"
  dw   → deletes to end of "foo" only; does NOT pull "bar" up (newline preserved)

count multiplication:
  3d2w  ==  d6w  ==  6dw   (must all produce identical results)

linewise self-application:
  3dd   deletes 3 lines starting at cursor line, cursor lands at first non-blank
        of the line that shifts into the cursor's old position
```

---

5 Text Objects

Text objects only ever appear as the target of an operator or in Visual mode; they are never standalone motions. `i` ("inner") excludes the delimiter/surrounding whitespace; `a` ("a", i.e. "around") includes it.

Object	`i` behavior	`a` behavior
`iw` / `aw`	current word, no surrounding space	word plus one side of adjacent whitespace

Object	`i` behavior	`a` behavior
`i( i) ib`	contents between nearest enclosing `( )`	includes the parens themselves
`i{ i} iB`	contents between nearest enclosing `{ }`	includes the braces
`i"` / `a"`	contents between nearest quote pair on the line	includes the quotes
`it` / `at`	contents between nearest enclosing HTML/XML tag pair	includes the tags
`ip` / `ap`	current paragraph (blank-line delimited)	paragraph plus one trailing blank line

Nesting matters for `i(`/`a(` etc.: with the cursor inside nested parentheses, the object is always the *nearest enclosing* pair, and `2i(` (a count before a text object) walks outward one level per count.

5.1 Text Object Resolution

Text object resolution is not one uniform algorithm — it is two, selected by object kind, and the spec should say so explicitly rather than imply a single flowchart covers every row of the table above.

Bracket and tag objects (`( ) b`, `{ } B`, `[ ]`, `< >`, `t`) resolve by nested balanced search:

1. If the cursor sits on the opening or closing delimiter itself, that pair is treated as the innermost enclosing pair.
2. Otherwise, search outward from the cursor for the nearest pair of matching delimiters that encloses the cursor's position; this search may cross line boundaries.
3. Apply the count: each additional count expands one level further outward (`2i(` means the pair enclosing the pair found in step 2).
4. Apply `i`/`a`: `a` includes the delimiters themselves; `i` excludes them and, in the documented special case where the interior spans multiple full lines on its own, also trims the newline immediately inside each delimiter, yielding a linewise rather than charwise range for that case specifically.

Quote objects (`i" a"`, `i' a'`, `i` a``) resolve differently, and this difference is not an oversight to reconcile with the bracket algorithm:

1. The search is restricted to the cursor's current line only — quotes are not treated as nesting across lines the way brackets do.
2. If the cursor is already inside a quoted span on that line, that span is the object.
3. If the cursor sits before the first quote on the line, Vim selects the *first* quoted string on the line, rather than searching backward from the cursor.
4. There is no expand-outward behavior for counts on quote objects, since there is nothing for a count to expand into — a line has no nesting of quoted spans in the sense brackets nest.

---

6 Rendering Mapping (renderer consumes parser state; never the reverse)

6.1 Fidelity Principle

The world exists solely to visualize parser state. Where visual intuition and actual Vim behavior conflict, Vim behavior wins, unconditionally — a world redesign to make some interaction feel more natural is never a license to change what a keystroke does. Data flows one way:

```
Parser → BufferModel / ParserState → Renderer
```

and never the reverse. No rendering-layer code may write to `ParserState`, `Buffer`, or `UndoNode`; the renderer's only inputs are read-only snapshots of parser output. This restates Section 0's second non-negotiable as a concrete data-flow rule specifically so that a violation is visible on sight in code review — a pull request containing a call from renderer code into parser or buffer state is wrong by inspection, without needing the Section 8 test suite to catch it at runtime.

Parser state	Visual representation
`mode: NORMAL`	full lighting, player mesh solid and idle-animated
`mode: INSERT`	scene desaturates ~30%, player mesh extends a "writing" appendage into the nearest surface, `hjkl` visibly do nothing when pressed (no camera response)
`mode: VISUAL_CHAR/LINE/BLOCK`	translucent selection volume grows live from `visualAnchor` to cursor; block mode renders as an actual rectangular prism, not a flat highlight
`pendingOperator ≠ null`	particle glow around player, color-coded per operator (`d` red, `y` blue, `c` yellow), persists until a motion/text-object resolves it or Esc cancels it
`countBuffer ≠ ""`	numeral rendered above player HUD-style, not in-world
`recordingMacro ≠ null`	a visibly rotating recording icon above player; on `@a` playback, the recorded action sequence visibly re-executes at accelerated but perceptible speed across each target in sequence
mark set (`ma`)	a small labeled beacon placed at that world position, persistent until overwritten
search active (`/pattern`)	all matching objects pulse/outline in the world simultaneously; `n/N` snaps camera focus to next/previous match in document order
undo/redo	the affected region visibly reverts/reapplies over ~200ms, not an instant cut, so the causal link between `u` and the specific change is legible

---

7 Level Data Format

```
{
  "id": "grammar-works-03",
  "title": "The Grammar Works — Room 3",
  "initialBuffer": ["The quick brown fox", "jumps over the lazy dog"],
  "initialCursor": { "line": 0, "col": 0 },
  "allowedCommands": ["h","l","j","k","w","b","e","d","y","c","p","P","0","$"],
  "goal": {
```

```
    "type": "bufferEquals",
    "target": ["quick brown", "jumps over the lazy dog"]
  },
  "par": 5,
  "hints": [
    { "afterKeystrokes": 15, "text": "You don't need to delete word by word." }
  ]
}
```

`allowedCommands` gates which keys the parser will accept in this level (unrecognized-but-otherwise-valid Vim commands should give an in-world "not yet unlocked" cue, not silently fail) — this is how the progressive teaching curve from the concept doc is actually enforced, rather than left as a level-design convention.

`goal.type` should support at minimum `bufferEquals` (exact match), `bufferMatches` (regex, for goals with multiple valid solutions), and `cursorAt` (motion-only levels where the buffer never changes).

7.1 Pedagogical Layer

`allowedCommands` implies a binary available/unavailable split; the actual teaching design needs a third state, and the three should not be implemented the same way:

```
type CommandAvailability = "unsupported" | "locked" | "unlocked";
```

- **`unsupported`** — not implemented by the parser in this version at all (see Section 10's roadmap). Typing it is a no-op with a generic not-available cue.
- **`locked`** — fully implemented and semantically correct in the parser, but withheld from the player by the current level's `allowedCommands`. Critically, a locked command must still be *parsed and resolved in full* by the engine, and only then have its resulting state change discarded in favor of a specific educational cue ("you'll unlock delete-word soon") — never short-circuited before parsing. Implementing it this way keeps the parser's grammar coverage complete and independently testable against Section 8's oracle even for commands no level has unlocked yet, rather than leaving locked commands as untested dead code until the level that first exposes them.

Gating happens at the resolved-command level, not at the keystroke level, and this is a decision rather than an implementation detail. Individual key-presses are never blocked or intercepted as they're typed — a player can freely type any sequence Section 1's grammar accepts, including the leading keys of a locked command, exactly as they could in real Vim. The check happens once a full `ResolvedCommand` (Section 3.3) exists: only then is its availability looked up, and only a `locked` or `unsupported` result triggers the discard-and-cue behavior described above. The alternative — refusing individual keystrokes that could only ever lead to a locked command — was considered and rejected, because it would mean the game's input handling diverges from Section 1's grammar depending on which level is loaded, which is exactly the kind of renderer/level-specific special-casing Section 6.1's Fidelity Principle rules out for rendering and should equally be ruled out for input handling. - **`unlocked`** — available, and behaves exactly per Sections 1–5, with no further distinction from ordinary Vim behavior.

---

8 Testing Plan

The parser module ships with a table-driven test suite before any rendering work starts:

```
interface ParserTestCase {
  initialBuffer: string[];
  initialCursor: Position;
  keys: string;              // raw keystrokes, e.g. "3dw"
  expectedBuffer: string[];
  expectedCursor: Position;
  expectedMode?: Mode;       // default NORMAL
}
```

The Section 4 "required test cases" become the seed set; coverage should expand to include every row of the Section 4 and Section 5 tables at least once, plus the count-multiplication identities as property-based tests (generate random `(a, b)` and assert `a*b da*b w` ≡ `d(a*b)w` for small values) rather than a handful of hand-picked examples.

8.1 Differential Testing Against Real Vim

Hand-written expected outputs, however careful, will not catch the edge cases that matter most, precisely because the person writing the test case and the person writing the parser share the same blind spots. The stronger design is an oracle: run the same buffer and keystrokes through real Vim semantics and assert Buffer World's parser agrees.

The concrete mechanism should be **Neovim's `--embed` RPC API**, not a shelled-out terminal Vim with output scraped from a pseudo-terminal. The RPC approach avoids ANSI/terminal-emulation fragility entirely and gives exact, structured buffer and cursor state directly:

```
interface OracleTestCase {
  initialBuffer: string[];
  initialCursor: Position;
  keys: string;   // raw keystrokes, exactly as Section 1's grammar would parse them
}

async function runOracle(tc: OracleTestCase): Promise<{ buffer: string[]; cursor:
↪  Position }> {
  // 1. spawn `nvim --embed --headless`
  // 2. attach via the msgpack-RPC client (the `neovim` npm package)
  // 3. nvim_buf_set_lines(0, 0, -1, false, tc.initialBuffer)
  // 4. nvim_win_set_cursor(0, [tc.initialCursor.line + 1, tc.initialCursor.col])  //
  ↪  nvim rows are 1-indexed
  // 5. nvim_input(tc.keys)
  // 6. read back nvim_buf_get_lines + nvim_win_get_cursor
  // 7. shut the embedded instance down
}
```

Given this harness, the test suite becomes generative rather than purely curated: run a large corpus of keystroke sequences — both the curated cases from Section 4/5 and randomly generated sequences over a range of seed buffers — through both `runOracle` and Buffer World's own parser, and assert equality. Any mismatch is either a genuine parser bug, or a deliberate, documented divergence where v1 intentionally doesn't implement something real Vim does (an `unsupported` command per Section 7.1 and Section 10). The harness needs a way to mark specific oracle mismatches as known and intentional rather than treating 100% oracle agreement as the bar for every possible input, since that bar is equivalent to reimplementing all of Vim.

This is a **Phase 0 requirement**, not a later hardening pass: Section 9's gate from Phase 0 to Phase 1 should require the oracle suite passing (net of documented divergences) alongside the hand-written Section 8 cases, since oracle testing is what catches the mismatches a spec document — including this one — cannot anticipate by construction.

9 Build Phases

The ordering below follows increasing abstraction rather than the order features were specified in this document: move, edit, select, transform, store, repeat, time-travel, create. Selection-and-transformation (`:s`/`:g`, Phase 4) is placed immediately after Grammar Works and before registers, marks, macros, or the undo tree, because it does not conceptually depend on any of them — it is a second grammar family alongside operators and text objects, not an advanced feature layered on top of the systems that follow it.

Phase	Deliverable	Gate to next phase
0	Headless parser + full Section 8 test suite passing, including the Section 8.1 oracle harness	100% of Section 4/5 required cases pass, and the oracle suite passes net of documented divergences
1	Three.js renderer, Plaza world only (`hjkl`, `0`, `$`, `gg`, `G`)	movement feels correct, no operators yet
2	Word Foundry + Editing Yards (`w/b/e`, `i/a/o/O`, Esc)	mode visualization (Section 6) implemented
3	Grammar Works (`d/y/c` × all Section 4 motions + Section 5 text objects)	full operator-motion grammar passes in-world
4	Searchworks (`:s`, `:%s`, `:g`/`:v` per Section 11)	Section 11's grammar passes its own test cases and oracle agreement, independent of registers/macros/undo
5	Archive Stacks (registers, marks, `Ctrl-w` splits as Z-axis)	Option A from the concept doc realized
6	Loop Chamber (macros, counts, `.` repeat)	macro replay visualization (Section 6) shipped
7	Undo Spire (Section 3.2 tree, `g-`/`g+`)	Section 3.2's tree structure and `g-`/`g+` ordering verified against the oracle
8	User-generated/editable levels (Section 12), using the Section 11 global/substitute grammar	leaderboard verification (13.3) passes against the Section 8.1 oracle harness

9.1 Searchworks

Searchworks is the first world built around a different mental model than everything before it. Plaza through Grammar Works are all *move-and-act-on- one-thing-at-a-time*; Searchworks introduces *select-a-population, then transform it*. The visual language should mark this shift rather than reuse the single-target glow from Section 6's operator visualization:

- A search or pattern (`/pattern`, or the search side of `:s`/`:g`) highlights every matching object across the visible world **simultaneously**, not one at a time — this is the population being selected, and it should read as a population, not a sequence of individually-found targets.
- A substitution or `:g`-driven action then visibly resolves that entire highlighted population toward a fixed point in one coordinated motion — every matching object transforming together — rather than replaying the same animation once per match in sequence, which

would misrepresent `:g`'s actual scan-then-act-once semantics from Section 11.1 as something closer to a loop the player is watching iterate.
- This is the concrete world-level realization of the match-set → operation → result model that Section 11 specifies formally: the world shows the match set as a visible population before any operation is applied, exactly so that the player can predict the result before committing to `:s` or `:g`, the same way Visual mode's selection volume (Section 6) lets a player preview an operator's effect before applying it.

Par scores and per-level best-keystroke tracking (Section 7's `par` field) can be persisted via the artifact storage API discussed for other projects (`window.storage`, keyed per level id) once this moves from a standalone prototype into a Claude-artifact build — worth deferring until Phase 1 is playable, since a persistence layer with no game to persist state for is premature.

---

10 Future Vim Coverage (Explicitly Out of Scope for v1)

This section exists so that a missing feature reads as a decision, not an oversight. Anything not named in Sections 1–8 or listed below as a future family is out of scope for v1, and a reviewer or contributor proposing it should check this list before assuming it was simply forgotten.

- **Window and tab management beyond Section 3's Z-axis splits** — `:tabnew`, tab navigation, and window-layout commands beyond the basic split model Phase 5 implements.
- **Replace mode (`R`)** — sticky multi-character replace, where typed characters overwrite existing ones until `Esc`, is excluded for v1. This is a deliberate exclusion, not an omission: Section 2's `Mode` type has no `REPLACE` state, and adding one would touch the state machine, the cursor invariants (Section 3.4), and the rendering mapping (Section 6) for a single mode family that isn't central to the curriculum this document specifies. This is distinct from single-character replace, `r<char>`, which is already in `<action>` (Section 1) and remains fully in scope — the two commands share a letter-adjacent name but not an implementation, and this bullet excludes only the sticky multi-character mode.
- **Folds** — `zf`, `zo`, `zc`, and fold-aware motions.
- **Additional text objects** — sentence objects (`is`/`as`), and any syntax- or treesitter-aware object beyond the fixed set in Section 5's table.
- **The Ex-command language** — beyond the two named exceptions below, full Ex-scripting, `:let`, arbitrary ranges beyond whole-buffer/global-matched lines, and anything requiring Vimscript evaluation is out of scope. **Two explicit exceptions**, specified in full in Section 11, exist because real usage showed the original single-command carve-out (macro replay only) was too narrow to cover how `:g` and `:s` actually get used together in practice: the global-command family (`:g/:global`, `:v/:g!/:vglobal`) combined with a small fixed set of per-line actions (`d`, `m`, `t/co`, `normal[!]`), and the substitute command (`:s`, `:%s`) with a bounded pattern/replacement grammar that explicitly excludes the `\=` expression register. These are two named additions with their own specification, not a reopening of the Ex-command family generally.
- **Plugins and Vimscript execution** — this project teaches Vim's built-in grammar; it is not a Vimscript runtime. This is also why the `\=` expression register is excluded from the `:substitute` grammar in Section 11 even though `:substitute` itself is in scope: `\=` evaluates an arbitrary expression (`printf(...)`, `strftime(...)`, `line('.')`) as part of the replacement, and is exactly the Vimscript-execution surface this exclusion is naming, just reachable through a single command's syntax rather than through a `:let`-style statement.
- **Treesitter-based or language-aware text objects** — anything requiring actual syntax parsing of the buffer's contents as a programming language, as opposed to the character-class

rules in Section 3.1.

- **Bash/readline "vi mode" (`set -o vi`) is not Vim and is not a coverage target at all**, named here only to close off a plausible confusion: it is a different program's emulation of Vim's modal editing, with real behavioral differences (no multi-line editing, a different and smaller text-object set) that would violate the Section 6.1 Fidelity Principle if treated as interchangeable with the actual Vim behavior this document specifies.

Each of the still-excluded items above is a plausible "Phase 9" for a version of this project that outgrows its original teaching scope, not a rejection of the idea — they are listed here specifically so that scope creep during Phases 0–8 has a named list to be checked against and deferred to, rather than being negotiated fresh each time it comes up.

---

11 Global Commands and Substitute

Section 10 named two exceptions to the Ex-command exclusion. This section specifies both in full, at the same level of rigor as Section 1, 4, and 5, since real usage showed both are load-bearing rather than peripheral.

11.1 The Global Command Family

```
<global>        ::= <global-normal> | <global-inverse>
<global-normal> ::= (":g" | ":global") "/" <pattern> "/" <line-action>
<global-inverse>::= (":v" | ":g!" | ":vglobal") "/" <pattern> "/" <line-action>
<line-action>   ::= "d"                              ; delete every matching line
                  | "m" <dest-line>                  ; move every matching line to
                  ↪  dest-line
                  | ("t" | "co") <dest-line>         ; copy every matching line to after
                  ↪  dest-line
                  | "normal!"? <keys>                ; run a keystroke sequence at each
                  ↪  match
                  | <substitute>                     ; see 12.2 — ":g/pat/s/.../.../ "
                  ↪  form
```

`<pattern>` is the same pattern grammar as Section 11.2's search side. The semantics that matter for correctness, and are easy to get wrong:

- `:g` first scans the **entire buffer once**, marking every line that matches, and only then executes `<line-action>` on each marked line in order — it does not re-scan after each action. This is why `:g/pattern/normal $3X` behaves correctly even when the action changes line lengths or content: the set of target lines was fixed before any action ran.
- `:g/^/m0` (move every line to line 0, in buffer order) is a real, idiomatic reverse-the-buffer idiom precisely because of the scan-then-act ordering above: each line, moved to position 0 in turn, ends up in reverse order relative to the original.
- `normal!` (with the bang) suppresses the user's own custom mappings during the per-line replay, using only the grammar this document specifies; `normal` without the bang is excluded from this grammar, since it would depend on a remapping layer this project does not implement.
- A `<keys>` sequence following `normal!` may itself invoke a macro register (`@a`) — this is the original narrow exception from the prior revision of Section 10 — but it is no longer the *only* supported `<line-action>`, since `d`, `m`, `t`/`co`, and direct keystrokes without a macro are equally common and equally in scope.
- **Nested `:g`/`:v` is illegal, matching real Vim.** Neither `<keys>` (following `normal!`) nor `<substitute>` may itself contain a `:g`, `:global`, `:v`, `:g!`, or `:vglobal` invocation. Real Vim

rejects this with `E147: Cannot do :global recursive`, and this grammar rejects it the same way: a parse-time error, not a silently-ignored or partially-applied command. This matters specifically because `<line-action>` already permits `normal!` with an arbitrary `<keys>` sequence, which is exactly general enough that an implementer could accidentally allow a second `:g` to slip through inside it if this exclusion weren't stated separately from the rest of the grammar.

11.2 Substitute

```
<substitute>  ::= <range>? "s" <sep> <pattern> <sep> <replacement> <sep> <flags>?
<range>       ::= "%"                    ; whole buffer — the primary case
                | <line-num> "," <line-num>
<sep>         ::= "/" | "#" | "!" | ...  ; any single non-alphanumeric char, consistent
↪  within one command
<flags>       ::= ("g" | "i")+           ; g: all matches per line, not just first; i:
↪  case-insensitive
```

No `<range>` means the current line only, not the whole buffer. The `?` on `<range>` in the grammar above is easy to misread as "range is optional, so omitting it is the same as the widest range" — this is backwards. A bare `:s/foo/bar/` with no range prefix applies to the cursor's current line only; `%` must be given explicitly to mean the whole buffer, exactly as `<range>` requires it as one of two named alternatives rather than as the grammar's default. This is worth stating outside the grammar because it is the single most common mistake a Vim beginner makes with `:s`, and a teaching tool that got it wrong would be teaching the mistake rather than correcting it.

Pattern grammar (search side), in scope:

Feature	Magic mode	Very-magic mode (`\v`)	Example from real usage
literal text	as written	as written	
any character	`.`	`.`	
quantifiers	`\+ \* \{n,m}`	`+ * {n,m}`	`\d\{1,3}`
character classes	`\d \w \s \u`	`\d \w \s \u`	`\d\{1,3}\.\d\{1,3}...`
anchors	`^ $`	`^ $`	
grouping	`\(...\)`	`(...)`	`\v^(a|g)`
alternation	`|` (needs `\`)	`|`	`\v^(a|g)`
match-start reset	`\zs`	`\zs`	`\v(^|[.!?] )\zs\w`

Both magic levels are in scope, not just one, because real usage mixes them (default-magic `\d\{1,3}` alongside very-magic `\v^(a|g)`), and a learner who only sees one style in-game would be underprepared for the other.

Replacement grammar, in scope: literal text, whole-match backreference `&`, numbered group backreferences `\0`–`\9`, and the case-modifier escapes `\u \U \l \L \e \E` (next-char, run-until-`\E`, and end-of-run case conversion — this is exactly the mechanism behind the small-caps and sentence-capitalization idioms real usage showed: `%s/\u\+/\L&/g` and `%s/\v(^|[.!?] )\zs\w/\u&/g`).

Explicitly excluded from the replacement grammar: the `\=` expression register (`\=printf(...)`, `\=strftime(...)`, `\=line('.')`, or any other expression). This is a single, deliberate cut: `\=` is the one point in `:substitute`'s own syntax where arbitrary Vimscript evaluation becomes reachable, and excluding it removes that whole surface at once without narrowing anything else in this section's grammar. A level or macro that depends on `\=` is out of scope by construction, the same as any other Vimscript dependency named in Section 10.

12 User-Generated and Editable Levels

The curriculum levels in Section 7 are hand-authored and pedagogically ordered. This section specifies a second, complementary level type: an arbitrary real editing task — reformat this data, fix this repeated typo, rename this pattern everywhere — imported and played as a level in its own right, scored the same way. This is the design's actual payoff: once a task can be expressed as a `before/after` buffer pair, the game doesn't need to know or care what the task was *for*.

12.1 Level Import Model

Import needs only what Section 7's `goal` field already accepts: a starting buffer and a target. Concretely:

- **Direct pair.** The author supplies `initialBuffer` and a target buffer directly; the target becomes a `bufferEquals` goal exactly as in Section 7.
- **Diff-based.** For large real files, pasting two full copies is unwieldy. The author instead supplies `initialBuffer` and a unified diff; the target buffer is computed once at import time by applying the diff, and only the resulting target is stored in the level definition. The diff itself is an authoring convenience — it is not part of the level's runtime representation, and nothing downstream (parser, oracle, leaderboard) ever sees it.

No new parser feature is required for either path; this section is an authoring workflow layered on Section 7's existing schema, not a new goal type.

12.2 Author-Plays-First Par Seeding

Section 7's `par` field assumed a level designer who simply knows the intended solution length. That doesn't generalize to arbitrary imported tasks, since computing a provably minimal keystroke sequence over the full grammar (counts, registers, macros, the `:g` exception above) is not tractable in general. Par is therefore seeded operationally rather than computed: **the author must solve the level themselves once at import time**, and their own keystroke count becomes the level's starting par. This guarantees, as a side effect of how the level is created, that at least one real solution exists — a level that turns out to be unsolvable as authored is caught at the moment of authoring, not after a player gets stuck.

12.3 Crowd-Sourced Par and Verified Replays

Par for an imported level is a discovered record, not a computed constant — the same posture Pipeline Golf takes toward transformation quality elsewhere in this line of work: no claimed optimum, only a current best that stands until beaten.

This only works if submissions are trustworthy, which means a submitted score can never be a bare number:

- Every submission stores the **actual keystroke sequence** that achieved it, not just its length.
- A submission is accepted only if replaying that exact sequence through the deterministic, headless parser (Sections 2–5) reaches the level's declared goal state in precisely the claimed number of keystrokes. This reuses the same replay mechanism Section 8.1 already requires for oracle testing — both are "replay these keys against this parser and check the result," differing only in which state they check against (real Neovim's output vs. a level's declared goal).

What counts as one keystroke is not obvious, and a par system built on an ambiguous definition isn't a par system. Two implementations that disagree on whether `f a` is one keystroke or two, or whether invoking a macro costs 1 keystroke or the N keystrokes it replays, will produce different par values for the *same* recorded solution — which defeats the purpose of a shared, cross-verifiable leaderboard. The rule is: **every distinct key-press event that reaches the parser as a discrete input counts as exactly one keystroke**, with no event ever counted as zero, as a fraction, or as a multiple of what was physically pressed. The table below resolves every case this document's grammar makes possible; the general rule above is what a new case should be checked against if one is missed.

Input	Keystroke count	Reasoning
A single command key (`d`, `w`, `x`, `~`, `$`, `%`, `{`, `.`)	1	One key-press event
`<Esc>`	1	One key-press event, same as any other key
A control-chord (`Ctrl-r`, `Ctrl-v`, `Ctrl-w`, `Ctrl-[`)	1	The chord is a single input event at the keyboard level; it is not counted per physical key held
Each digit of a count prefix (`12dw`)	1 per digit	`12dw` = `1`,`2`,`d`,`w` = **4**, not 3 — a two-digit count costs exactly as much as physically typing two digits, which is what makes a needlessly large count an honest cost rather than a free multiplier
`f{char}`, `F{char}`, `t{char}`, `T{char}`, `r{char}`	2	The command key and the target character are separate key-press events
Each character of a search pattern, including the leading `/` or `?` and the terminating `<CR>`	1 per character	`/foo<CR>` = `/`,`f`,`o`,`o`,`<CR>` = **5**
Each character of an Ex command line, including the leading `:` and the terminating `<CR>`	1 per character	`:%s/a/b/g<CR>` = **12** — this is what makes the par system fairly weigh "type a substitute command" against "edit by hand," rather than giving `:s` an unearned discount
Each typed character of inserted text, plus the mode-entering key and the terminating `<Esc>`	1 per character/key	`ihi<Esc>` = `i`,`h`,`i`,`<Esc>` = **4**
A text object or motion spelled with multiple characters (`iw`, `aw`, `ge`)	1 per character	`diw` = `d`,`i`,`w` = **3**
Macro recording (`qa ... q`)	1 for `q`, 1 for the register letter, 1 for each keystroke performed while recording, 1 for the terminating `q`	Recording has no discount: it costs exactly what was typed, including the `q`/register/`q` bracketing
Macro invocation (`@a`)	2	`@` and the register letter, regardless of how many keystrokes the macro replays internally — this is the entire reason macros are efficient in this accounting, not an oversight to patch

Input	Keystroke count	Reasoning
Repeated macro invocation (`10@a`)	1 per digit, plus 2	`10@a` = `1`,`0`,`@`,`a` = **4**, however many keystrokes the macro itself replays each of the ten times
`.` (dot-repeat)	1	Regardless of the size or complexity of the command being repeated — this is symmetric with macro invocation: both are cases where one keystroke stands for an arbitrarily larger replayed effect

A worked example ties this to Section 11.1's own idiom: `:g/^\d\+$/d` — every character including `:`, `g`, `/`, `^`, `\`, `d`, `+`, `$`, `/`, `d`, and the terminating `<CR>` counts individually, for a total of **11** keystrokes, regardless of how many lines in the buffer actually match and get deleted. This is the same principle as the Ex-command row above, applied to `:g` rather than `:s`: the cost is what was typed, not what the command's effect turned out to be.

- Leaderboards are naturally shared, per-level data: `window.storage` with `shared: true`, keyed by level id, storing an ordered list of `{keystrokes: string, playerLabel: string}` entries rather than scores alone, so any entry remains independently re-verifiable by replaying it — including, as a spot check, by running it through the Section 8.1 oracle as well as the level's own goal check.

12.4 Repetition Detection and Macro-Friendliness

Real editing tasks that are worth turning into levels are disproportionately the ones with repeated structure — the same shape of edit recurring at many sites — which is exactly the condition under which a short macro dominates any manual approach. This is the actual reason this pivot makes macros the star mechanic rather than one system among several.

At import time, a lightweight structural diff between `initialBuffer` and the target can detect approximately-repeated edit patterns and annotate the level with an estimated repetition count. This is metadata for hinting and level browsing ("macro-friendly," tagged with an approximate multiplicity), not a gameplay gate — `allowedCommands` and the lock system from Section 7.1 still govern what's actually available in a given level. Detection only decides what the level is labeled, not what it permits.

12.5 Placement in the Build Plan

This section depends on the full grammar being solid — specifically macros (Phase 6) and, for large imported files, the undo-tree work (Phase 7) — and benefits from Section 8.1's oracle harness already existing, since the same replay-and-verify mechanism does double duty as the leaderboard's anti-cheat check. It belongs after the curriculum phases as its own **Phase 8**, so that level-authoring tooling never becomes a dependency the core teaching path in Phases 0–7 has to wait on.